

Cheatsheet

Contents

- Links

[FastAPI](#) is a modern, fast web framework for building APIs with Python 3.8+ based on standard Python type hints. Created by Sebastián Ramírez in 2018, **FastAPI** is built on top of Starlette for web parts and Pydantic for data parts. It provides automatic API documentation, high performance due to its async capabilities, and strong type checking.



👉 New to [App-Generator](#)? Sign IN with [GitHub](#) or [Generate Web Apps](#) in no time (free service).

Basic Route Setup and Path Operations

```
from fastapi import FastAPI, Path, Query
from typing import Optional

app = FastAPI()

# Basic routes
@app.get("/")
async def root():
    return {"message": "Hello World"}

# Path parameters
@app.get("/items/{item_id}")
async def read_item(
    item_id: int = Path(..., title="Item ID", ge=1),
    q: Optional[str] = Query(None, max_length=50)
):
    return {"item_id": item_id, "q": q}

# Query parameters
@app.get("/search/")
async def search_items(
    q: str = Query(..., min_length=3),
    skip: int = Query(0, ge=0),
    limit: int = Query(10, le=100)
):
    return {"q": q, "skip": skip, "limit": limit}

# Multiple HTTP methods
@app.post("/items/")
async def create_item(item: Item):
    return item

@app.put("/items/{item_id}")
async def update_item(item_id: int, item: Item):
    return {"item_id": item_id, **item.dict()}
```



Request Body and Data Models

```
from pydantic import BaseModel, Field, EmailStr
from typing import Optional, List

# Basic model
class Item(BaseModel):
    name: str
    price: float
    is_offer: bool = False
    tags: List[str] = []

# Advanced model with validation
class User(BaseModel):
    username: str = Field(..., min_length=3, max_length=50)
    email: EmailStr
    full_name: Optional[str] = None
    age: int = Field(..., ge=0, le=120)

    class Config:
        schema_extra = {
            "example": {
                "username": "johndoe",
                "email": "john@example.com",
                "full_name": "John Doe",
                "age": 25
            }
        }

# Nested models
class Order(BaseModel):
    items: List[Item]
    user: User
    total: float

# Using models in routes
@app.post("/users/")
async def create_user(user: User):
    return user

@app.post("/orders/")
async def create_order(order: Order):
    return order
```



Dependencies and Dependency Injection

```
from fastapi import Depends, HTTPException
from typing import Annotated

# Basic dependency
async def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Class-based dependency
class CommonQueryParams:
    def __init__(self, q: Optional[str] = None, skip: int = 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

# Using dependencies
@app.get("/items/")
async def read_items(
    commons: Annotated[CommonQueryParams, Depends(CommonQueryParams)],
    db: Annotated[Session, Depends(get_db)]
):
    items = db.query(Item).offset(commons.skip).limit(commons.limit)
    if commons.q:
        items = items.filter(Item.name.contains(commons.q))
    return items

# Dependency with sub-dependencies
async def verify_token(token: str = Depends(oauth2_scheme)):
    user = get_user(token)
    if not user:
        raise HTTPException(status_code=401)
    return user

@app.get("/users/me")
async def read_user_me(current_user: User = Depends(verify_token)):
    return current_user
```



Authentication and Security

```
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm
from jose import JWTError, jwt
from passlib.context import CryptContext

# Password hashing
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# OAuth2 setup
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="token")

# JWT token creation
def create_access_token(data: dict):
    to_encode = data.copy()
    expires = datetime.utcnow() + timedelta(minutes=15)
    to_encode.update({"exp": expires})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

# Login endpoint
@app.post("/token")
async def login(form_data: OAuth2PasswordRequestForm = Depends()):
    user = authenticate_user(form_data.username, form_data.password)
    if not user:
        raise HTTPException(status_code=401)
    access_token = create_access_token(data={"sub": user.username})
    return {"access_token": access_token, "token_type": "bearer"}

# Protected endpoint
@app.get("/protected")
async def protected_route(token: str = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        username: str = payload.get("sub")
        if username is None:
            raise HTTPException(status_code=401)
    except JWTError:
        raise HTTPException(status_code=401)
    return {"message": "Welcome to protected resource"}
```



Error Handling and Custom Responses

```
from fastapi import HTTPException, status
from fastapi.responses import JSONResponse, RedirectResponse
from fastapi.exceptions import RequestValidationError
from fastapi.encoders import jsonable_encoder

# Custom exception handler
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(request, exc):
    return JSONResponse(
        status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
        content=jsonable_encoder({"detail": exc.errors()})
    )

# Raising exceptions
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    if item_id > 100:
        raise HTTPException(
            status_code=404,
            detail="Item not found",
            headers={"X-Error": "Item error"},
        )
    return {"item_id": item_id}

# Custom responses
@app.get("/redirect", response_class=RedirectResponse)
async def redirect():
    return "https://fastapi.tiangolo.com"

# Response model
@app.get("/users/{user_id}", response_model=UserOut)
async def read_user(user_id: int):
    return get_user(user_id)
```



Background Tasks and WebSockets

```
from fastapi import BackgroundTasks
from fastapi.websockets import WebSocket

# Background task
def write_log(message: str):
    with open("log.txt", mode="a") as log:
        log.write(f"{message}\n")

@app.post("/send-notification/{email}")
async def send_notification(
    email: str,
    background_tasks: BackgroundTasks
):
    background_tasks.add_task(write_log, f"Notification sent to {email}")
    return {"message": "Notification sent in background"}

# WebSocket endpoint
@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    try:
        while True:
            data = await websocket.receive_text()
            await websocket.send_text(f"Message received: {data}")
    except WebSocketDisconnect:
        print("Client disconnected")
```



Middleware and CORS

```
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware

# CORS setup
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Custom middleware
@app.middleware("http")
async def add_process_time_header(request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time)
    return response

# Compression middleware
app.add_middleware(GZipMiddleware)
```

Database Integration (SQLAlchemy)

```
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

# Database setup
SQLALCHEMY_DATABASE_URL = "postgresql://user:password@localhost/dbname"
engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

# Model
class DBUser(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    email = Column(String, unique=True, index=True)
    hashed_password = Column(String)

# CRUD operations
@app.post("/users/", response_model=User)
def create_user(user: UserCreate, db: Session = Depends(get_db)):
    db_user = DBUser(**user.dict())
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user
```



Testing

© [App Generator](#) - [v0.0.136](#)


```
from fastapi.testclient import TestClient
import pytest

client = TestClient(app)

# Basic test
def test_read_main():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello World"}

# Test with parameters
def test_create_item():
    response = client.post(
        "/items/",
        json={"name": "Test Item", "price": 45.5}
    )
    assert response.status_code == 200
    assert response.json()["name"] == "Test Item"

# Async test
@pytest.mark.asyncio
async def test_websocket():
    client = TestClient(app)
    with client.websocket_connect("/ws") as websocket:
        websocket.send_text("Hello")
        data = websocket.receive_text()
        assert data == "Message received: Hello"
```



API Documentation and OpenAPI

```
from fastapi.openapi.utils import get_openapi

# Custom OpenAPI schema
def custom_openapi():
    if app.openapi_schema:
        return app.openapi_schema
    openapi_schema = get_openapi(
        title="My API",
        version="1.0.0",
        description="This is a very custom OpenAPI schema",
        routes=app.routes,
    )
    app.openapi_schema = openapi_schema
    return app.openapi_schema

app.openapi = custom_openapi

# Tags for grouping
@app.get("/users/", tags=["users"])
async def read_users():
    return [{"username": "Rick"}, {"username": "Morty"}]

# API metadata
app = FastAPI(
    title="My Super API",
    description="This is a super fancy API",
    version="1.0.0",
    terms_of_service="http://example.com/terms/",
    contact={
        "name": "Support",
        "email": "support@example.com"
    },
    license_info={
        "name": "Apache 2.0",
        "url": "https://www.apache.org/licenses/LICENSE-2.0.html"
    }
)
```



Pro Tips

- Use Pydantic models for request/response validation
- Implement proper error handling with custom exception handlers
- Use dependency injection for code reuse and testing
- Implement proper logging
- Use async functions when dealing with I/O operations
- Utilize FastAPI's automatic API documentation
- Implement proper security measures (authentication/authorization)
- Use proper status codes and response models
- Write comprehensive tests
- Keep routes organized using APIRouter

- Use background tasks for time-consuming operations
- Implement rate limiting for public APIs

Links

- 👉 New to [App-Generator](#)? Join our 10k+ Community using GitHub [One-Click SignIN](#).
- 👉 Download [products](#) and start fast a new project
- 👉 Bootstrap your startUp, MVP or Legacy project with a [custom development](#) sprint

